HW1 Profiling Analysis Report

Keshav Goel

2024101051

1. EXECUTIVE SUMMARY

This report analyzes the performance bottlenecks of two programs:
log_analyzer (Part A) and image_filter (Part B).

- Part A suffers from excessive dynamic memory allocation (malloc/free) in its hot path and inefficient string handling functions (strcmp, strtok).

- Part B is severely limited by poor memory access patterns (column-major traversal) causing a 59% cache miss rate, and a logical error where a constant initialization function is re-executed for every pixel.

- Recommendation: Prioritize optimizing Part B. It offers a higher theoretical speedup (~2.78x) compared to Part A (~1.82x) and the fixes involve correcting loops and hoisting invariant code, which yields high returns for low effort.

==========
2. PROFILING TOOL COMPARISON
==========

Feature	gprof	perf	Callgrind
Method	Instrumentation (Function calls) + Sampling (Time)	Event Sampling (Hardware counters)	CPU Emulation / Instrumentation
Overhead	Medium	Low (1-5%)	Very High (50x-100x)
Visibility	User-code only (mostly) Libraries	User + Kernel + (User code)	Instruction level
Strength	Call counts, simple setup	Low impact, finds system bottlenecks	Exact call graphs, finding logical errors

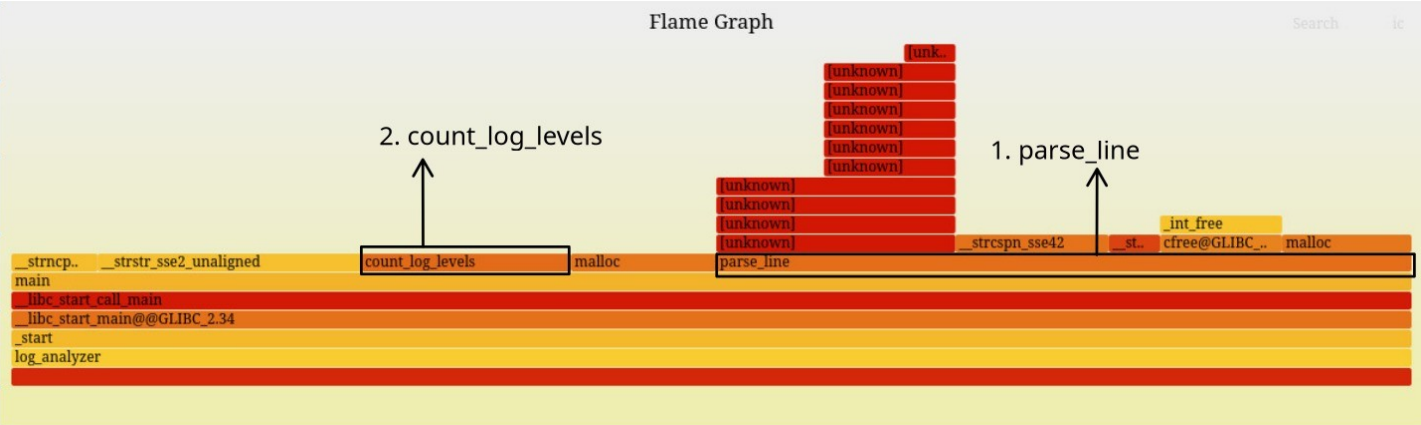
==========
3. HOTSPOT ANALYSIS
==========

PART A: LOG ANALYZER

The perf report identified parse_line as the primary bottleneck (~50% overhead). Deeper analysis shows the time is fragmented across libc functions:

• __strstr_sse2_unaligned (~19%)
• __strncpy_evex (~6%)
• malloc/free overhead (~9% each, found via Callgrind)
• count_log_levels (~15%)

Evidence: The perf hierarchy shows parse_line dominating execution, while Callgrind explicitly links the high instruction count to the allocation loop within parse_line.



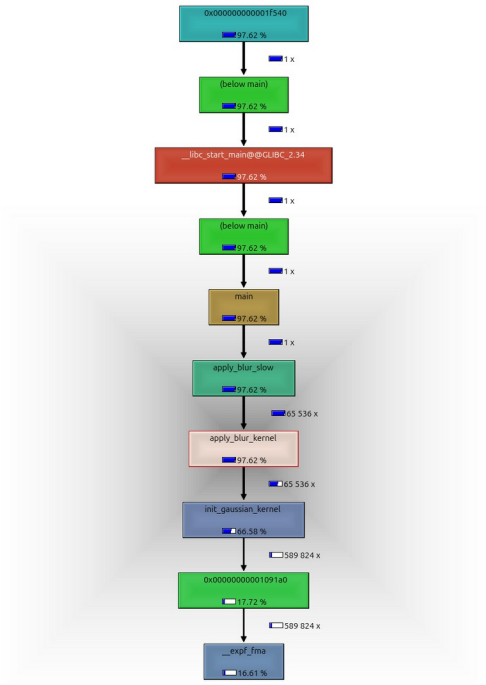
(Annotation: The wide "plateau" in the center represents parse_line and its children, indicating where the CPU spends most of its time. The tall tower on the left shows __strstr_sse2_unaligned called by search_messages.)

PART B: IMAGE FILTER

Two major issues were identified:

1. Cache Thrashing: apply_blur_slow iterates x (width) then y (height). Since C arrays are row-major, this strides through memory, evicting cache lines before they are fully used. This results in a 59.14% cache miss rate.

2. Redundant Computation: apply_blur_kernel calls init_gaussian_kernel for every single pixel. This constant calculation consumes ~48% of the total instructions, despite the kernel values never changing.



==========
4. AMDAHL'S LAW RESULTS
==========

• Part A: Optimizing the parse_line hotspot (~50% of time) by 10x yields a total program speedup of ~1.82x.

Calculation: Speedup = 1 / ((1 - 0.50) + (0.50 / 10))
= 1 / (0.50 + 0.05)
= 1 / 0.55
= 1.82x

• Part B: Optimizing the blur hotspots (~80% of time) by 5x yields a total program speedup of ~2.78x.

Calculation: Speedup = 1 / ((1 - 0.80) + (0.80 / 5))
= 1 / (0.20 + 0.16)
= 1 / 0.36
= 2.78x

==========
5. OPTIMIZATION RECOMMENDATIONS
==========

1. Fix Part B Loop Order (Priority 1):
Swap the loops in apply_blur_slow to iterate y then x. This will drastically reduce cache misses from 59% to near-zero for sequential access.

2. Hoist Part B Kernel Init (Priority 2):

Move init_gaussian_kernel out of the loop in apply_blur_kernel. Calculate
it once and pass it in. This eliminates ~48% of redundant instructions.

3. Optimize Part A Allocation (Priority 3):
Move the temporary buffer allocation in parse_line to main (reuse one buffer)
to eliminate malloc/free overhead per line (~18% of execution time).

==========